Universidad del Valle de Guatemala
Ingeniería en Ciencias de la Computación
Curso: Programación orientada a objetos



Análisis y diseño programa
Lab 3 - Herencia

Roger Yorkaef Méndez García - 26786
Laura Suseth Morales Murga - 26281

09/10/2026

Análisis

1. Requisitos funcionales

Información que debe conservar el sistema

- Envíos: código, remitente, destinatario, peso real, destino, categoría, estado e intentos de entrega.
- Datos específicos: dimensiones de los paquetes, valor declarado de los paquetes frágiles y temperatura de las cargas refrigeradas.
- Vehículos: placa, capacidad máxima y disponibilidad de refrigeración.
- Despachos: vehículo asignado, envíos incluidos y estado del despacho.
- Ingresos: cobros por registro, cobros por reintentos e ingresos por categoría.

Operaciones que debe permitir

1. Cotizar envíos, calculando su costo y tiempo estimado sin registrarlos.
2. Registrar envíos y cobrar su costo inicial.
3. Registrar vehículos.
4. Crear despachos y asignar o retirar envíos.
5. Registrar la salida de los despachos.
6. Registrar entregas exitosas o fallidas.
7. Consultar envíos y vehículos.
8. Generar reportes de envíos, ingresos y despachos activos.

Condiciones de validez

- Los códigos de envío y las placas deben ser únicos y no estar vacíos.
- Los valores numéricos deben ser válidos y respetar los límites establecidos.
- Los sobres no pueden superar 0.5 kg.
- Las cargas refrigeradas deben tener una temperatura entre -25 °C y 8 °C y destino urbano.
- Los envíos asignados no pueden superar la capacidad del vehículo.
- Las cargas refrigeradas requieren vehículos con refrigeración.
- Un envío no puede pertenecer a dos despachos simultáneamente.
- Solo pueden asignarse envíos que estén en bodega y tengan intentos disponibles.
- Un vehículo no puede tener dos despachos sin cerrar.
- Un despacho vacío no puede salir.

- Los envíos deben respetar el máximo de intentos permitido según su categoría.
- Las operaciones inválidas deben rechazarse sin modificar la información ni los ingresos.
- Los envíos solo pueden retirarse de despachos en preparación.
- Los envíos que agotan sus intentos quedan devueltos al remitente y no pueden asignarse nuevamente.
- Cada reintento adicional genera un cargo de Q15 cuando sale el despacho.
- Un envío en bodega solo puede asignarse si no está reservado en otro despacho. Para controlar esto, se utiliza un atributo booleano llamado asignado.
- Antes de asignar un envío, se debe comprobar su disponibilidad, peso real, capacidad del vehículo y compatibilidad. Si alguna condición no se cumple, la operación se rechaza sin modificar los datos.
- Los resultados de entrega no pueden registrarse dos veces para un mismo envío dentro del mismo despacho.

2. Decisiones que dependen de la categoría del envío

- La clase padre Envio contiene la información común, mientras que las clases hijas se encargan de las decisiones específicas de cada categoría.
- Cálculo del costo: Cada clase calcula su propia tarifa. Sobre utiliza un costo fijo, Paquete considera el peso facturable, PaqueteFragil agrega embalaje y seguro, y CargaRefrigerada incluye el cargo por cadena de frío.
- Tiempo de entrega: Cada categoría determina los días hábiles según el destino. Los paquetes frágiles agregan un día al tiempo de un paquete normal.
- Intentos de entrega: Sobre y Paquete permiten tres intentos, PaqueteFragil permite dos y CargaRefrigerada solamente uno.
- Compatibilidad con vehículos: Cada clase determina si el vehículo es adecuado. Las cargas refrigeradas requieren vehículos con refrigeración, mientras que las demás categorías pueden utilizar cualquier vehículo.
- Validaciones específicas: Cada clase verifica sus restricciones, como el peso máximo de los sobres, las dimensiones de los paquetes y la temperatura y destino de las cargas refrigeradas.

3. Estados de los envíos y despachos

Estados de los envíos

- En bodega: El envío queda en este estado cuando se registra o cuando se retira de un despacho en preparación. También regresa a bodega si falla una entrega y todavía tiene intentos disponibles.
- Para diferenciar un envío disponible de uno reservado, se utiliza el atributo booleano asignado. Un envío puede estar en bodega, pero no estar disponible porque ya pertenece a un despacho en preparación.
- En ruta: El envío cambia a este estado cuando sale el despacho al que fue asignado.
- Cuando se registra la salida del despacho, el envío cambia a EN_RUTA y su cantidad de intentos aumenta en uno.
- Entregado: El envío pasa a este estado cuando se registra una entrega exitosa.
- Devuelto al remitente: El envío llega a este estado cuando agota sus intentos de entrega y ya no puede asignarse a otro despacho.

Estados de los despachos

- En preparación: El despacho se crea con un vehículo y permite asignar o retirar envíos mientras no haya salido.
- En ruta: El despacho cambia a este estado cuando sale con al menos un envío. A partir de ese momento se registran los resultados de entrega.
- Cerrado: El despacho queda cerrado cuando todos sus envíos tienen un resultado registrado. En ese momento, el vehículo vuelve a estar disponible.
- El sistema verifica que cada envío tenga un único resultado registrado dentro del despacho. Cuando todos los envíos tienen un resultado, el despacho se cierra y el vehículo queda disponible nuevamente.

4. Clases y sus responsabilidades

Clase Envio (abstracta)

Atributos

Atributo	**Tipo**	**Visibilidad**
codigo	String	private
remitente	String	private
destinatario	String	private
pesoReal	double	private
destino	String	private
estado	String	private
intentos	int	private
asignado	boolean	private

Métodos

Método	**Retorno**	**Visibilidad**
Envio(codigo: String, remitente: String, destinatario: String, pesoReal: double, destino: String)	Constructor	protected
calcularCosto()	double	public abstract
estimarEntrega()	int	public abstract
maxIntentos()	int	public abstract
esCompatible(vehiculo: Vehiculo)	boolean	public
registrarSalida()	void	public
registrarResultado(entregado: boolean)	void	public
getCodigo()	String	public
getPesoReal()	double	public
getEstado()	String	public
getIntentos()	int	public
estaDisponible()	boolean	public
asignar()	void	public
retirarAsignacion()	void	public

Clase Sobre

Atributos
No tiene atributos adicionales.

Métodos

Método	**Retorno**	**Visibilidad**
Sobre(codigo: String, remitente: String, destinatario: String, pesoReal: double, destino: String)	Constructor	public
calcularCosto()	double	public
estimarEntrega()	int	public
maxIntentos()	int	public

Clase Paquete

Atributos

Atributo	**Tipo**	**Visibilidad**
largo	double	private
ancho	double	private
alto	double	private

Métodos

Método	**Retorno**	**Visibilidad**
Paquete(codigo: String, remitente: String, destinatario: String, pesoReal: double, destino: String, largo: double, ancho: double, alto: double)	Constructor	public
calcularPesoFacturable()	double	public
calcularCosto()	double	public
estimarEntrega()	int	public
maxIntentos()	int	public

Clase PaqueteFragil

Atributos

Atributo	**Tipo**	**Visibilidad**
valorDeclarado	double	private

Métodos

Método	**Retorno**	**Visibilidad**
PaqueteFragil(codigo: String, remitente: String, destinatario: String, pesoReal: double, destino: String, largo: double, ancho: double, alto: double, valorDeclarado: double)	Constructor	public
calcularCosto()	double	public
estimarEntrega()	int	public
maxIntentos()	int	public

Clase CargaRefrigerada

Atributos

Atributo	**Tipo**	**Visibilidad**
temperatura	double	private

Métodos

Método	**Retorno**	**Visibilidad**
CargaRefrigerada(codigo : String, remitente: String, destinatario: String, pesoReal: double, destino: String, temperatura: double)	Constructor	public
calcularCosto()	double	public
estimarEntrega()	int	public

maxIntentos()	int	public
esCompatible(vehiculo: Vehiculo)	boolean	public

Clase Vehiculo

Atributos

Atributo	**Tipo**	**Visibilidad**
placa	String	private
capacidadMaxima	double	private
refrigerado	boolean	private
despachoActivo	Despacho	private

Métodos

Método	**Retorno**	**Visibilidad**
Vehiculo(placa: String, capacidadMaxima: double, refrigerado: boolean)	Constructor	public
getPlaca()	String	public
getCapacidadMaxima()	double	public
tieneRefrigeracion()	boolean	public
estaDisponible()	boolean	public
asignarDespacho(despacho: Despacho)	void	public
liberar()	void	public

Clase Despacho

Atributos

Atributo	**Tipo**	**Visibilidad**
vehiculo	Vehiculo	private
envios	ArrayList<Envio>	private
estado	String	private
resultadosRegistrados	ArrayList<Envio>	private

Métodos

Método	**Retorno**	**Visibilidad**
Despacho(vehiculo: Vehiculo)	Constructor	public
asignarEnvio(envio: Envio)	boolean	public
retirarEnvio(envio: Envio)	boolean	public
salir()	boolean	public
registrarResultado(envio : Envio, entregado: boolean)	boolean	public
calcularPeso()	double	public
getEstado()	String	public
getVehiculo()	Vehiculo	public
capacidadDisponible()	double	public
getEnvios()	ArrayList<Envio>	public
cerrar()	void	private

Clase SistemaEnvios (Controlador)

Atributos

Atributo	**Tipo**	**Visibilidad**
envios	ArrayList<Envio>	private
vehiculos	ArrayList<Vehiculo>	private
despachos	ArrayList<Despacho>	private
ingresosRegistro	double	private
ingresosReintentos	double	private

Métodos

Método	**Retorno**	**Visibilidad**
SistemaEnvios()	Constructor	public
cotizar(envio: Envio)	double	public
registrarEnvio(envio: Envio)	boolean	public

registrarVehiculo(vehiculo: Vehiculo)	boolean	public
crearDespacho(vehiculo: Vehiculo)	Despacho	public
consultarEnvios()	ArrayList<Envio>	public
consultarVehiculos()	ArrayList<Vehiculo>	public
generarReportes()	String	public

Clase VistaConsola (Vista)

Atributos

Atributo	**Tipo**	**Visibilidad**
scanner	Scanner	private
sistema	SistemaEnvios	private

Métodos

Método	**Retorno**	**Visibilidad**
VistaConsola(sistema: SistemaEnvios)	Constructor	public
mostrarMenu()	void	public
solicitarOpcion()	int	private
mostrarMensaje(mensaje: String)	void	public
iniciar()	void	public
cotizarEnvio()	void	private
registrarEnvio()	void	private
registrarVehiculo()	void	private
gestionarDespachos()	void	private
consultarDatos()	void	private
mostrarReportes()	void	private
leerEntero(mensaje: String)	int	private
leerDouble(mensaje: String)	double	private

Clase Main (Inicio)

Métodos

Método	**Retorno**	**Visibilidad**
main(args: String[])	void	public static

5. Justificación de la relación entre Paquete y PaqueteFragil

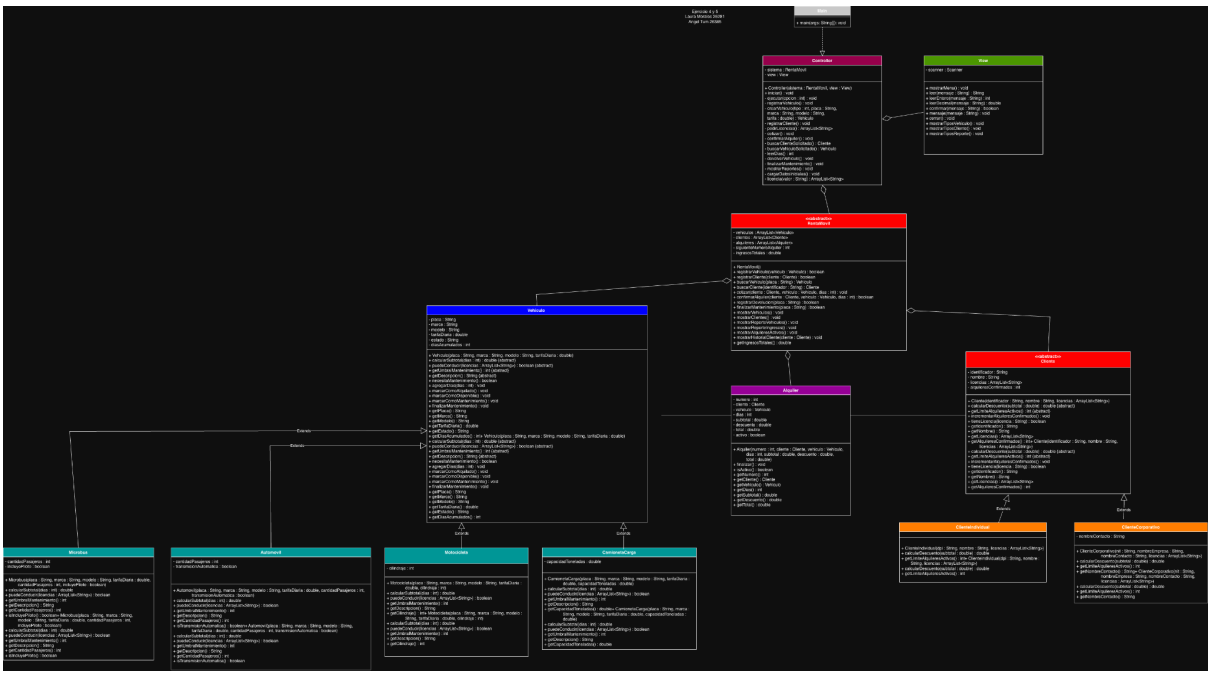
- La clase PaqueteFragil hereda de Paquete porque comparte sus características, como el peso, las dimensiones y el cálculo del peso facturable. Sin embargo, agrega un valor declarado para calcular el seguro y un cargo adicional por embalaje.
- Esta relación permite reutilizar los atributos y métodos de Paquete, sobrescribiendo únicamente los métodos que tienen reglas diferentes, como el costo, el tiempo de entrega y los intentos permitidos.
- Una alternativa sería que PaqueteFragil heredara directamente de Envio, pero esto obligaría a repetir los atributos y cálculos de un paquete normal. Por eso, heredar de Paquete evita duplicar código y facilita el mantenimiento del programa.

6. Incorporación de nuevas categorías

- Para agregar una nueva categoría de envío, como mascotas por ejemplo, se tiene que crear una clase hija de Envio que implemente sus propias reglas de costo, tiempo de entrega, compatibilidad con vehículos e intentos permitidos.
- No sería necesario modificar las clases existentes ni la lógica de los despachos y reportes, ya que estas trabajan con objetos de tipo Envio mediante polimorfismo.
- Únicamente se tendría que actualizar el menú para permitir registrar y cotizar la nueva categoría. Esto facilita ampliar el sistema sin afectar su funcionamiento actual.

Enlace del UML:

CLICK



Enlace Repositorio GitHub:

hola rogelio yorkaef